Assignment II

Distributed and Cloud Computing

Chapter 9

Emerging Trends in Distributed and Cloud Computing

Rushal Manandhar

790333

BE. Computer

VII semester

Date : 9 August, 2026

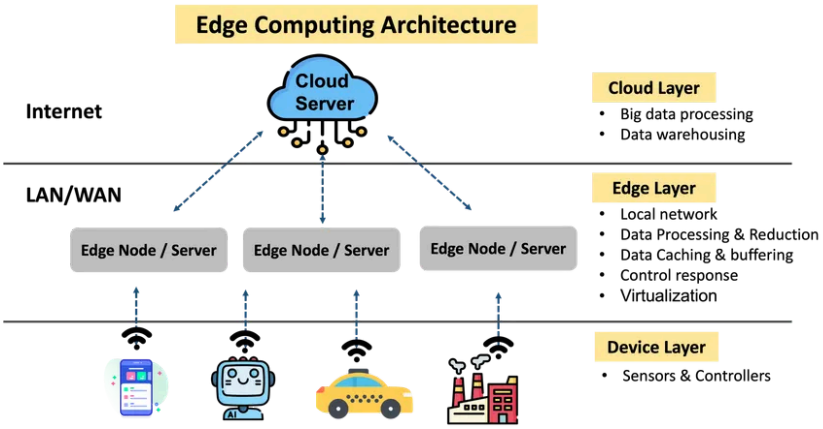
Table Of Content

Edge Computing and Fog Computing.....2
1.1 Why they exist.....2
1.2 Edge Computing.....2
1.3 Fog Computing.....3
1.4 Edge vs Fog vs Cloud — Comparison.....4
Serverless Architecture.....4
2.1 What "serverless" actually means.....4
2.2 Core Model: Functions-as-a-Service (FaaS).....4
2.3 How it works.....5
2.4 Key Characteristics.....5
2.5 Advantages.....5
2.6 Challenges.....7
2.7 Serverless vs Traditional Cloud Servers.....7
Docker and Kubernetes.....9
3.1 Docker — Containerization.....9
3.2 Kubernetes (K8s) — Container Orchestration.....9
3.3 Docker vs Kubernetes — how they relate.....11
3.4 Real-world usage today (2026).....11
Cloud-Native and Microservices.....12
4.1 Cloud-Native — Definition.....12
4.2 Microservices Architecture.....13
4.3 How it all connects.....14
4.4 Real-world usage today (2026).....14
Conclusion.....16

Edge Computing and Fog Computing

1.1 Why they exist

Traditional cloud computing sends all data from devices to a centralized data center for processing. As IoT devices, sensors, and smart systems grew massively in number, this created problems: high latency, heavy bandwidth use, and poor performance for time-sensitive applications (e.g., autonomous vehicles, industrial automation). Edge and fog computing solve this by moving computation **closer to where data is generated**.



Figure：Edge computing architecture overview
Source：The research team

1.2 Edge Computing

Definition: Processing data directly on or very near the device that generates it (sensors, smartphones, IoT gateways, cameras) — at the "edge" of the network, instead of sending it to a distant cloud server.

Key characteristics:

- Computation happens at or near the data source (local devices, edge servers, routers).
- Minimal reliance on network connectivity to a central cloud.
- Extremely low latency — often single-digit milliseconds.
- Reduces bandwidth usage since raw data isn't constantly sent to the cloud.

Examples: Self-driving cars processing camera/sensor data instantly; smart security cameras detecting motion locally; industrial sensors on a factory floor triggering immediate shutdowns.

Advantages:

- Real-time / near-instant response
- Reduced bandwidth cost
- Better privacy (sensitive data can be processed locally without leaving the device)
- Works even with intermittent internet connectivity

Challenges:

- Limited compute/storage power at the edge
- Harder to manage and update thousands of distributed edge devices
- Security risks if edge devices are physically accessible/tampered with

1.3 Fog Computing

Definition: An intermediate computing layer that sits **between** the edge devices and the centralized cloud. Fog nodes (local servers, gateways, routers with processing capability) handle data that doesn't need to go all the way to the cloud, but is too much for a single edge device to process alone.

Key characteristics:

- Acts as a "middle layer" in a hierarchical architecture: Device (Edge) → Fog Node → Cloud
- Aggregates data from multiple edge devices before deciding what to send to the cloud
- Provides more processing power than a single edge device, but still local (e.g., within a building, campus, or city)

Examples: A smart city system where fog nodes aggregate traffic data from many intersections before sending summarized insights to the cloud; a factory's local server coordinating multiple machines.

Advantages:

- Reduces the amount of raw data sent to the cloud (bandwidth savings at scale)
- Better scalability than pure edge computing (can manage many devices)
- Still much lower latency than sending everything to a distant cloud

1.4 Edge vs Fog vs Cloud — Comparison

Aspect	Edge Computing	Fog Computing	Cloud Computing
Location	On/near the device	Local network (LAN) layer	Centralized, remote data centers
Latency	Lowest	Low	Highest
Processing power	Limited	Moderate	Very high
Scope	Single device/small group	Multiple devices/local area	Global
Use case	Instant local decisions	Regional data aggregation	Big data storage, heavy analytics

Serverless Architecture

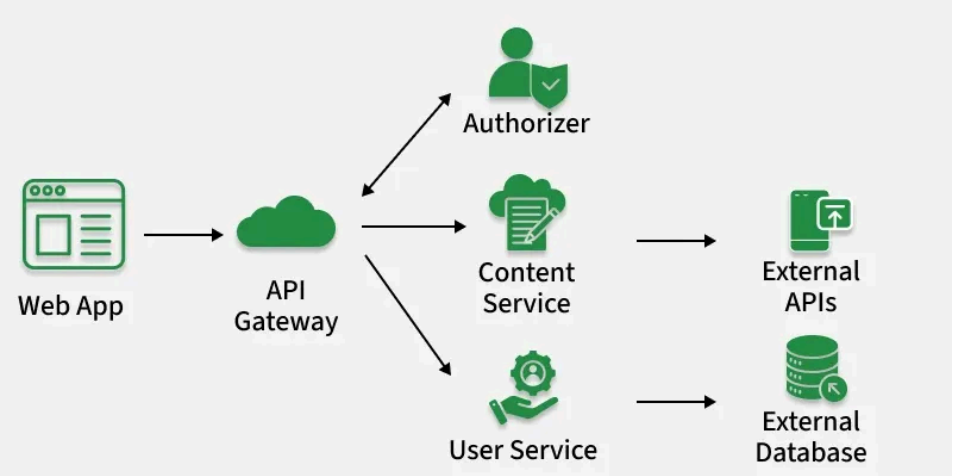
2.1 What "serverless" actually means

Serverless does **not** mean there are no servers — servers still run the code. It means the **developer doesn't manage or provision servers**. The cloud provider automatically handles infrastructure, scaling, and maintenance. Developers just write and deploy code (functions), and it runs on demand.

2.2 Core Model: Functions-as-a-Service (FaaS)

The most common form of serverless computing. Code is broken into small, independent **functions** that execute in response to specific **triggers/events** (e.g., an HTTP request, a file upload, a database update, a scheduled timer).

Examples of platforms: AWS Lambda, Azure Functions, Google Cloud Functions.



2.3 How it works

1. Developer writes a function and deploys it to the platform.
2. The function stays idle (no cost) until triggered by an event.
3. When triggered, the provider automatically spins up the required resources, runs the function, and returns the result.
4. Resources are torn down after execution — the provider handles scaling up or down automatically, even to zero.

2.4 Key Characteristics

- **Event-driven:** functions execute only in response to triggers.
- **Auto-scaling:** the platform scales instances up or down (including to zero) automatically based on demand.
- **Pay-per-execution:** billing is based on actual compute time used (often measured in milliseconds), not on pre-allocated server time. You don't pay when nothing is running.
- **Statelessness:** each function execution is independent and typically doesn't retain data between calls (state must be stored externally, e.g., in a database).
- **No infrastructure management:** provisioning, patching, and scaling servers is entirely the provider's responsibility.

2.5 Advantages

- Cost-efficient for variable or unpredictable workloads (no idle server cost)
- Automatic scaling without manual configuration
- Faster development — teams focus purely on business logic, not infrastructure
- Reduced operational overhead (no server maintenance)

2.6 Challenges

- **Cold start latency:** if a function hasn't run recently, spinning it up from idle takes extra time, hurting response speed.
- **Vendor lock-in:** serverless functions are often tightly coupled to a specific provider's ecosystem, making migration difficult.
- **Debugging/monitoring complexity:** distributed, short-lived functions are harder to trace and test than traditional long-running applications.
- **Not ideal for long-running or heavy compute tasks** (most platforms enforce execution time limits, e.g., 15 minutes on AWS Lambda).

2.7 Serverless vs Traditional Cloud Servers

Aspect	Traditional Server (IaaS/PaaS)	Serverless (FaaS)
Server management	You manage/provision	Fully managed by provider
Scaling	Manual or configured autoscaling	Automatic, including to zero
Billing	Pay for allocated server time	Pay only per execution
Idle cost	Yes, servers run even when unused	None
Best for	Long-running, predictable workloads	Event-driven, sporadic workloads

Docker and Kubernetes

3.1 Docker — Containerization

Definition: Docker is a platform that packages an application along with everything it needs to run (code, libraries, dependencies, system tools, runtime) into a single unit called a **container**. A container behaves the same way regardless of where it's deployed — laptop, test server, or cloud — solving the classic "it works on my machine" problem.

Key concepts:

- **Image:** A read-only blueprint/template of an application and its environment.
- **Container:** A running instance of an image — lightweight and isolated, but shares the host machine's OS kernel (unlike a full virtual machine).
- **Dockerfile:** A text file with instructions to build a Docker image.

- **Docker Hub:** A public registry to store and share container images.

Why containers instead of VMs: Virtual machines virtualize entire hardware and need a full guest OS each, making them heavy (GBs, minutes to boot). Containers virtualize only at the OS level, so they're lightweight (MBs, boot in seconds) and far more efficient for running many isolated apps on one machine.

3.2 Kubernetes (K8s) — Container Orchestration

Definition: Kubernetes is an open-source system (originally built by Google) for **automating the deployment, scaling, networking, and management of containers** across a cluster of machines. Once an application is split into many containers (common in microservices), manually managing them becomes impossible at scale — Kubernetes automates this.

Core functions:

- **Orchestration:** Decides which machine (node) runs which container.
- **Auto-scaling:** Automatically increases or decreases the number of running containers based on load.
- **Self-healing:** Automatically restarts or replaces containers that crash or fail health checks.
- **Load balancing:** Distributes network traffic evenly across containers.
- **Rolling updates & rollbacks:** Updates applications with zero downtime, and can revert instantly if something breaks.
- **Service discovery:** Containers can find and communicate with each other automatically, even as they're created/destroyed.

Key building blocks:

- **Pod:** Smallest deployable unit in Kubernetes — one or more tightly coupled containers running together.
- **Node:** A physical or virtual machine that runs pods.
- **Cluster:** A set of nodes managed together by Kubernetes.
- **Deployment:** A configuration describing how many replicas of a pod should run and how to update them.

3.3 Docker vs Kubernetes — how they relate

They are **not competitors** — they work together at different layers:

Aspect	Docker	Kubernetes
Role	Builds and runs individual containers	Manages/orchestrates many containers across machines
Scope	Single machine	Entire cluster of machines
Handles scaling	No (manual)	Yes (automatic)
Handles failure recovery	No	Yes (self-healing)
Analogy	The shipping container itself	The port/logistics system managing thousands of containers

3.4 Real-world usage today (2026)

Container and Kubernetes adoption has moved from "cutting-edge" to "standard enterprise infrastructure":

- <cite index="7-1">92% of organizations now run containers in production, with Kubernetes as the dominant orchestrator</cite>, and <cite index="1-1">Kubernetes holds roughly a 92% share of the container orchestration tools market</cite>.
- <cite index="1-1">82% of container-using organizations run Kubernetes in production, up from 80% in 2024 and 66% in 2023, while 98% of surveyed organizations report adopting cloud-native techniques in some form</cite>.
- <cite index="1-1">77% of Fortune 100 companies now run Kubernetes in production, and the average enterprise operates over 6 Kubernetes clusters</cite>.

- <cite index="1-1">Kubernetes adoption skews heavily toward large organizations — 91% of users come from companies with over 1,000 employees</cite>, since running and maintaining clusters requires dedicated engineering expertise.
- <cite index="7-1">Over 5.6 million developers worldwide now use Kubernetes, a 67% increase since 2020</cite>.

Where it's mostly used:

- **Tech / streaming platforms:** e.g. <cite index="2-1">Spotify runs more than 4,000 microservices across roughly 200 Kubernetes clusters, serving over 600 million monthly active users</cite>.
- **Banking, finance, and e-commerce:** for handling high, unpredictable traffic reliably.
- **Healthcare and government systems:** increasingly adopting it for scalable, resilient infrastructure — <cite index="2-1">Kubernetes has become standard infrastructure across industries from banking and healthcare to government agencies</cite>.
- **AI/ML infrastructure:** growing fast, as Kubernetes is increasingly used to schedule and scale AI training/inference workloads.

In short: Docker is how you package an app; Kubernetes is how you run it reliably at scale. Together they form the backbone of most modern cloud deployments.

Cloud-Native and Microservices

4.1 Cloud-Native — Definition

Cloud-native refers to designing and building applications specifically to take full advantage of cloud computing — rather than simply moving an old-style application onto cloud servers ("lift-and-shift"). Cloud-native apps are built to be scalable, resilient, and manageable using modern practices like containers, microservices, dynamic orchestration, and automation from the start.

Core principles / pillars of cloud-native:

- **Microservices** – breaking an app into small, independent services (see below).
- **Containers** – packaging each service consistently (Docker).
- **Dynamic orchestration** – automated deployment/scaling (Kubernetes).
- **DevOps & CI/CD** – continuous integration/delivery pipelines for fast, frequent releases.
- **Declarative APIs / Infrastructure-as-Code** – infrastructure is defined and managed through code/config, not manual setup.

4.2 Microservices Architecture

Definition: An architectural style where an application is built as a collection of small, independent services, each responsible for one specific business capability (e.g., a "user service," "payment service," "notification service"), communicating with each other over the network (typically via APIs).

This contrasts with the older **monolithic architecture**, where the entire application (UI, business logic, database access) is built and deployed as a single, tightly coupled unit.

Key characteristics:

- Each microservice can be developed, deployed, updated, and scaled **independently**.
- Services often have their own database (avoiding a single shared bottleneck).
- Communication typically happens via lightweight protocols — REST APIs, gRPC, or message queues.
- Teams can use different languages/technologies for different services ("polyglot" development).

Monolith vs Microservices

Aspect	**Monolithic**	**Microservices**
Codebase	Single unified codebase	Multiple independent services
Deployment	Deploy entire app for any change	Deploy only the changed service
Scaling	Scale the whole app together	Scale individual services as needed
Failure impact	One failure can crash the whole app	Failure is isolated to one service
Development	Simple initially, hard to manage at scale	More complex setup, but scales better with large teams

Advantages:

- Independent, faster deployments (small teams own small services)
- Fault isolation — one failing service doesn't necessarily bring down the whole system
- Easier to scale only the parts of the system under heavy load
- Flexibility in technology choice per service

Challenges:

- Increased operational complexity (many services to monitor, deploy, secure)
- Network latency and reliability issues between services
- Requires strong DevOps practices, monitoring, and orchestration (this is exactly why microservices and Kubernetes are so commonly used together)
- Data consistency across services is harder than with one shared database

4.3 How it all connects

Cloud-native, microservices, Docker, and Kubernetes form one connected ecosystem: **Cloud-native** is the overall design philosophy → **Microservices** is the architectural style used to build the app → **Docker** packages each microservice into a container → **Kubernetes** orchestrates, scales, and manages all those containers in production.

4.4 Real-world usage today (2026)

Microservices and cloud-native design are now the default approach for building modern, large-scale applications rather than a niche trend:

- <cite index="5-1">Cloud-native adoption now touches 98% of organizations, with 93% of them actively using or evaluating Kubernetes</cite> as the platform to run these architectures.
- <cite index="6-1">Growth in the Kubernetes/cloud-native market is being driven in part by rising demand for microservices, alongside hybrid cloud growth and the need for agility</cite>.
- <cite index="5-1">Experimental adoption is over — cloud-native has become the standard foundation of enterprise IT</cite>, meaning most new large-scale software projects are designed cloud-native and microservices-based from the start rather than being converted later.

Where it's mostly used:

- **Large-scale consumer platforms:** streaming (Spotify, Netflix), e-commerce (Amazon), ride-sharing, and social media — where different features (search, recommendations, payments, chat) scale independently and change frequently.
- **Banking and fintech:** isolating critical services (payments, fraud detection) so failures don't take down the entire platform.
- **SaaS companies:** enabling frequent, independent feature releases without redeploying the whole product.
- **Enterprises undergoing "digital transformation":** migrating legacy monolithic systems into cloud-native, microservices-based architectures to improve scalability and release speed.

In short: Cloud-native + microservices is now the mainstream way large, scalable applications are architected — and Docker + Kubernetes are the practical tools that make running them at scale possible.

Conclusion

These four topics are deeply interconnected pieces of the modern distributed and cloud computing landscape. Edge/fog computing addresses where data is processed, serverless architecture addresses how code execution and infrastructure are managed, and Docker/Kubernetes together with cloud-native/microservices address how applications are built, packaged, and scaled reliably. Collectively, they represent the shift from centralized, monolithic, manually-managed systems toward distributed, automated, and highly scalable computing — which now defines the majority of modern enterprise IT infrastructure.

Reference

1. Bonomi, F., et al. (2012). "Fog computing and its role in the Internet of Things." Proceedings of the first edition workshop on Mobile cloud computing (MCC). ACM.
2. Shi, W., et al. (2016). "Edge computing: Vision and challenges." IEEE Internet of Things Journal, 3(5), 637-646.
3. Red Hat – "What is serverless?" https://www.redhat.com/en/topics/cloud-native-apps/what-is-serverless
4. FOSS Post. Kubernetes Adoption Rate Statistics 2026: Enterprise Usage And Container Adoption. https://fosspost.org/kubernetes-adoption-rate-statistics/
5. DEV Community. Companies Using Kubernetes in 2026: Who Runs K8s and How They Scale. https://dev.to/matheus_releaserun/companies-using-kubernetes-in-2026-who-runs-k8s-and-how-they-scale-2log
6. ReleaseRun. Kubernetes Statistics and Adoption Trends in 2026. https://releaserun.com/kubernetes-statistics-adoption-2026/
7. Docker Inc. *Docker Documentation — What is a Container?* https://docs.docker.com/get-started/docker-overview/
8. Kubernetes.io (CNCF). *Kubernetes Documentation — Concepts Overview.* https://kubernetes.io/docs/concepts/overview/